Dictionary and Tolerant Retrieval

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
Master Degree in Computer Engineering

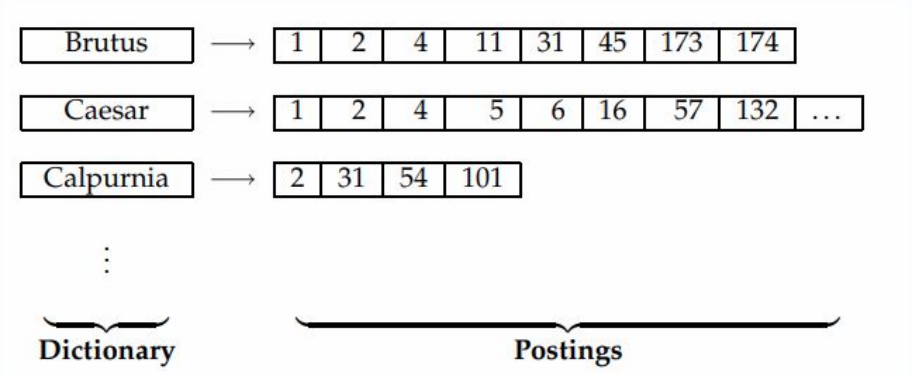
Data Structure for Dictionaries

Structures for Dictionaries

In previous sections we have been talking about different implementation of Inverted Index.

Here instead different data structures for dictionaries are described.

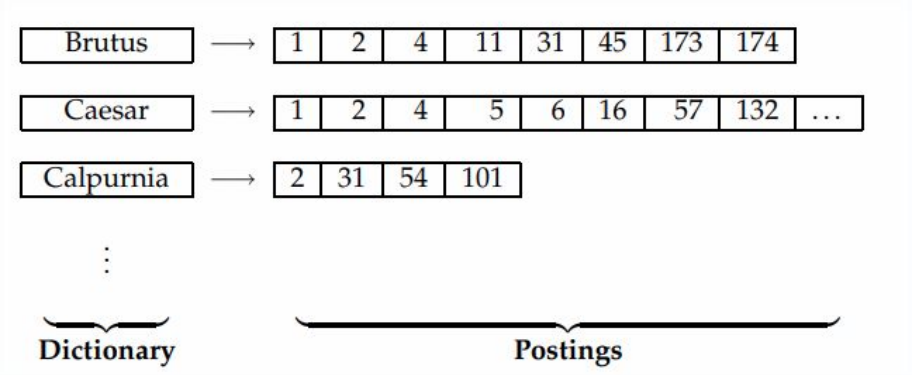
Usually, two different data structures are used: **hashing** and **search tree**.



Hashing

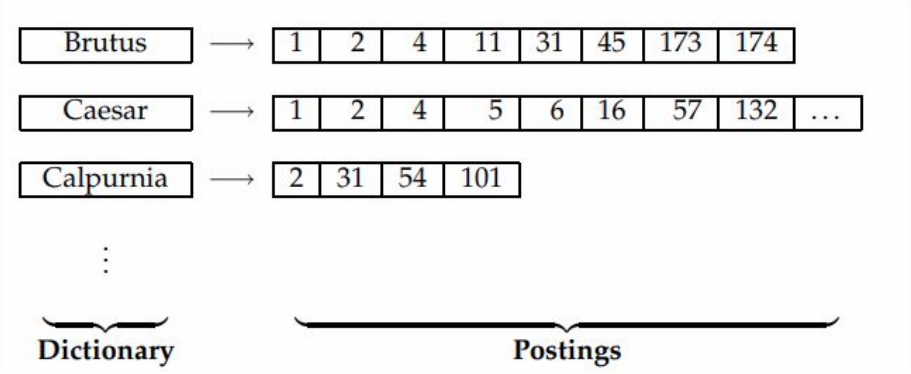
With the Hashing Data Structure, during indexing, each term in the Vocabulary is hashed into an integer large enough, **such that hash collisions are unlikely**

At **Query Time**, each term is hashed and then we get the Postings List by using the hash value.



Considerations with Hashing

- little **variation** in the term (even adding an accent) changes the hash value
- it is not possible to search for all terms beginning with a **prefix**
- for **small Dictionaries**, hashing may work fine, but for big Dictionary (like the Web) it might not be a good choice.



Search Trees

Search Trees overcome many of the issues introduced by Hashing.

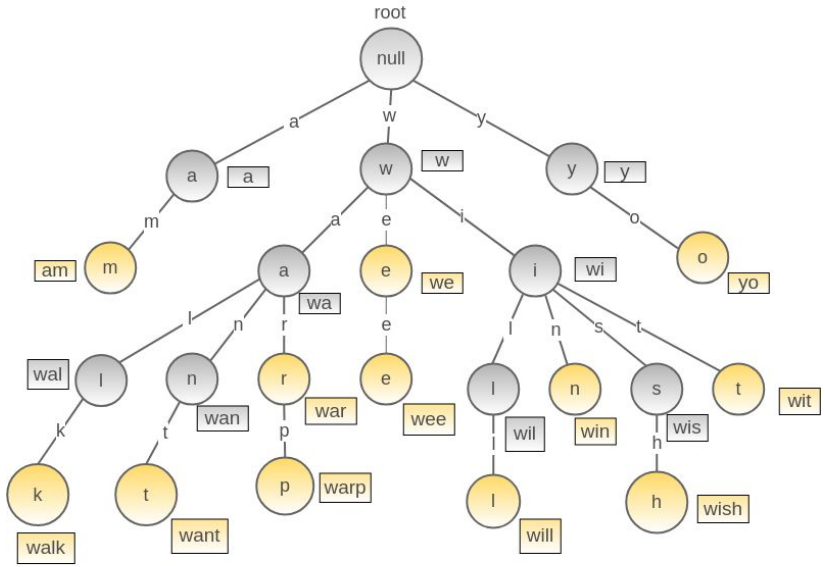
For instance they permit to enumerate all terms beginning with **a prefix**

There are many implementations of Search Trees:

- Prefix Tree (also known as Trie)
- Prefix Binary Tree
- Prefix B-Tree

Prefix Tree

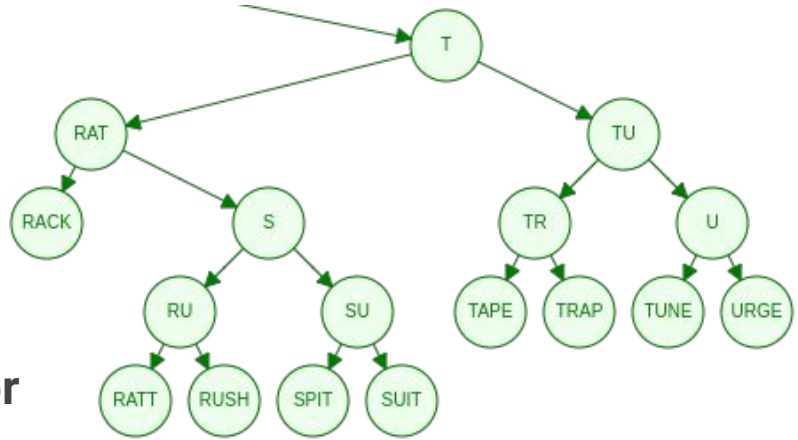
- It is a type of **k-ary** search tree
- Keys are strings
- **Searched Terms** can be in the leaves or in the internal nodes
- A search operation can go **one way or the others**, depending on the value to search for
- Big problems:
  - the tree is not **balanced**
  - the arity is **26^+ (if not just letters are considered)**



Word(s) with prefix **wa**: walk, want, war, warp
Word(s) with prefix **we**: wee
Word(s) with prefix **wi**: will, win, wish, wit

Prefix Binary Tree

- It is a **binary tree**
- Keys are **strings**
- Each **internal node** represent a test (no real terms in the collection)
- The **leaves** represent the terms we can search for
- A search operation can go **one way or the other**
- Big problems:
  - the tree is not **balanced**
  - **auxiliary terms** must be introduced



Prefix Binary Tree: Separator (1)

To build a proper Prefix Binary Tree, we need to introduce the meaning of **Separator**.

Let $\mathbf{S_j}$ be a term already stored in the Tree. Let $\mathbf{S_i}$ be the new term to be stored into the tree as a **left child** of S_j.

A separator S is the <u>shortest</u> string derived from S_j such that $S_i < S <= S_j$

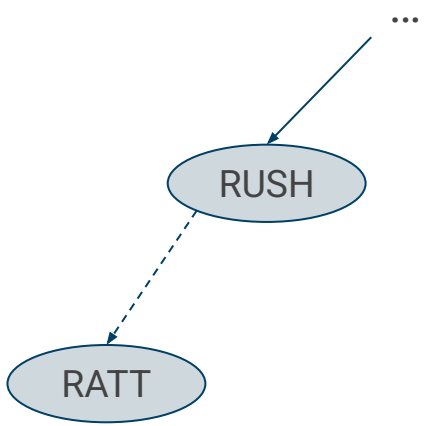
Before inserting S_i into the tree, the Separator must be inserted and the Tree must be rearranged

Prefix Binary Tree: Separator - Example 1

Let RUSH be a term already stored in the Tree.

Let RATT the new term to be stored into the tree

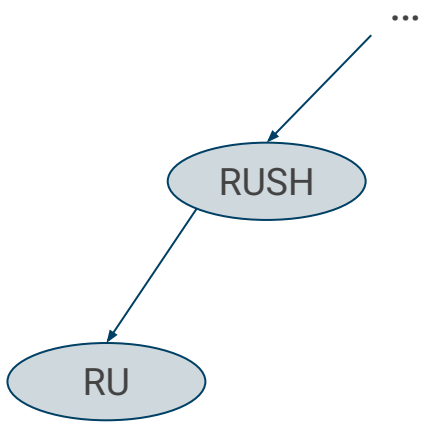
Being RATT < RUSH, RATT will be inserted as the left child of RUSH



Prefix Binary Tree: Separator - Example 1

Before inserting RATT, we need:

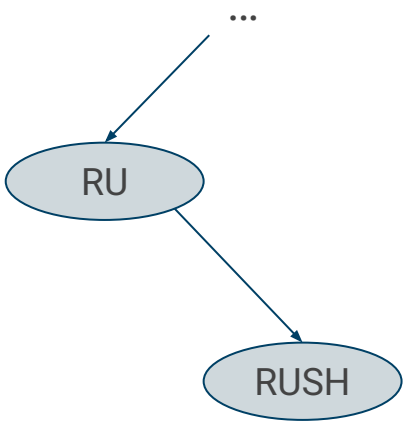
1. Defining the selector S, here RU
2. Adding RU as left child of RUSH



Prefix Binary Tree: Separator - Example 1

Before inserting RATT, we need:

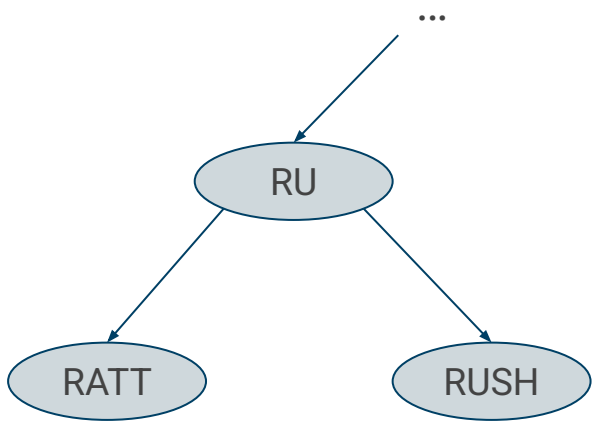
1. Defining the selector S, here RU
2. Adding RU as left child of RUSH
3. Rotating the subtree, pivoting on RUSH



Prefix Binary Tree: Separator - Example 1

Before inserting RATT, we need:

1. Defining the selector S, here RU
2. Adding RU as left child of RUSH
3. Rotating the subtree, pivoting on RUSH
4. Inserting the new term RATT



Prefix Binary Tree: Separator (2)

To build a proper Prefix Binary Tree, we need to introduce the meaning of **Separator**.

Let $\mathbf{S_i}$ be a term already stored in the Tree. Let $\mathbf{S_j}$ be the new term to be stored into the tree as a **right child** of S_i.

A separator S is the shortest string derived from S_i such that $S_i < S <= S_j$

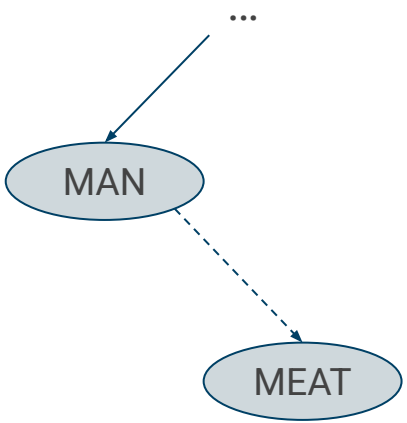
Before inserting S_j into the tree, the Separator must be inserted and the Tree must be rearranged

Prefix Binary Tree: Separator - Example 2

Let MAN be a term already stored in the Tree.

Let MEAT the new term to be stored into the tree

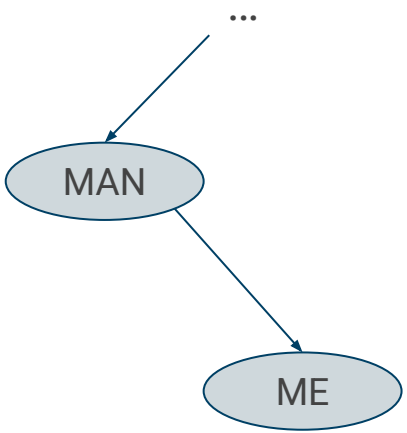
Being MEAT > MAN, MEAT will be inserted as the righty child of MAN



Prefix Binary Tree: Separator - Example 2

Before inserting MEAT, we need:

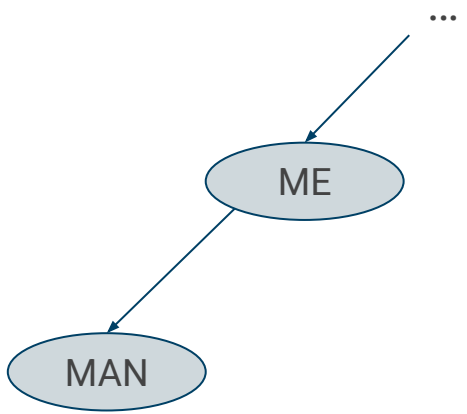
1. Defining the selector S, here ME
2. Adding ME as right child of MAN



Prefix Binary Tree: Separator - Example 2

Before inserting MEAT, we need:

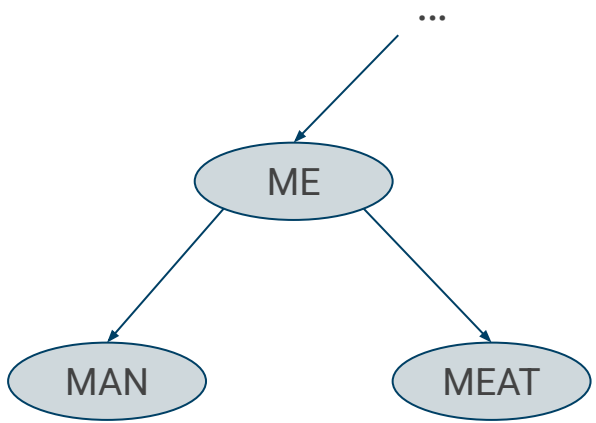
1. Defining the selector S, here ME
2. Adding ME as right child of MAN
3. Rotating the subtree, pivoting on MAN



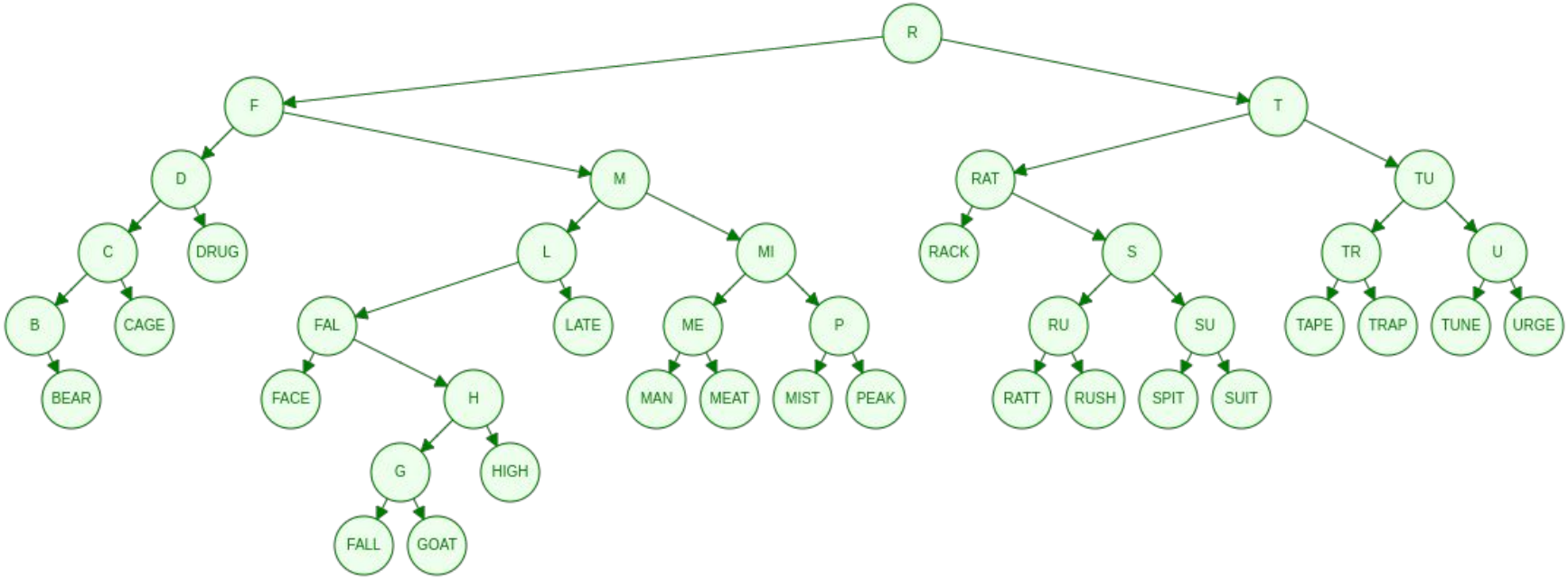
Prefix Binary Tree: Separator - Example 2

Before inserting MEAT, we need:

1. Defining the selector S, here ME
2. Adding ME as right child of MAN
3. Rotating the subtree, pivoting on MAN
4. Inserting the new term MEAT

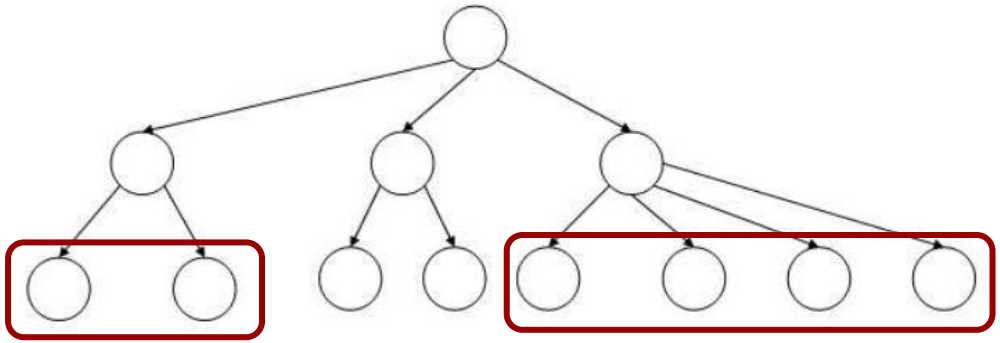


Prefix Binary Tree



Prefix B-Tree

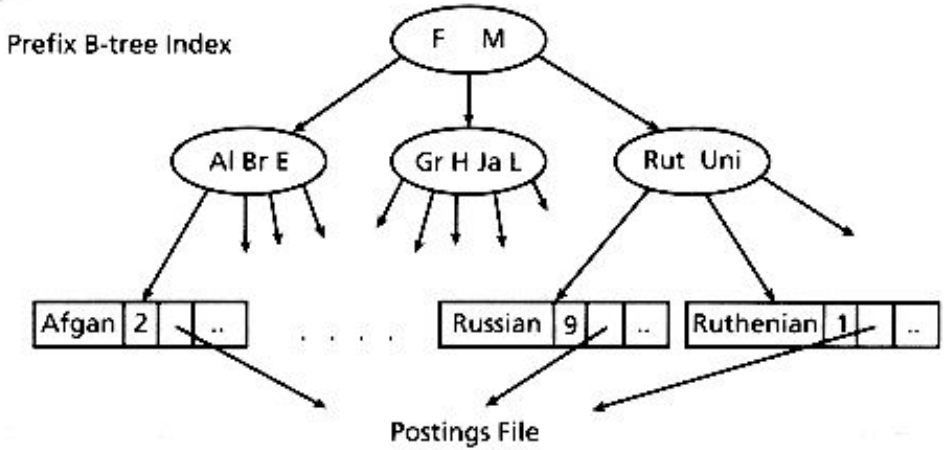
- It solves the problem of **rebalancing** tree
- It satisfies all the properties already seen for the **traditional B-Tree** (for example the **minimum degree** and others)



Prefix B-Tree

Other properties for the Prefix B-Tree

- Leaves contains the terms to store in the dictionary
- Internal nodes represent a test for a sequence of characters
- The strings in the **internal nodes** are named **separators** and are derived the terms



Prefix B-Tree: separator

Definition of **separator**:

If S_{m-1} and S_m are two keys in a full node such that S_m is the **median key** and S_{m-1} is its predecessor, a separator S is the **shortest string derived** from S_m such that

$$\mathbf{S_{m-1} < S <= S_m}$$

Prefix B-Tree: separator

For example: we have a **full leaf** so we need to split it into two nodes.

The median value is MET, the predecessor is EXPOSE. We can use M as the shortest string derived from MET.

The separator M is added to the leaf parent and the leaf is splitted

Prefix B-Tree: separator

Of course there are several separators that can be used but it is more appropriate to select the shortest separator possible because **shorter separator reduce the height and the size** of the B-Tree

Prefix B-Tree: an example

This is the list of keys we want to insert into our B-Tree: *rack, face, meat, tune, mist, late, ratt, hear, drug, peak, suit, fall, spit, cage, high, bear, goat, trap, urge, rush, tape*

Let consider a B-Tree with a minimum degree = 2 (this means the number of keys in each node ranges between 1 and 3)

Prefix B-Tree: an example

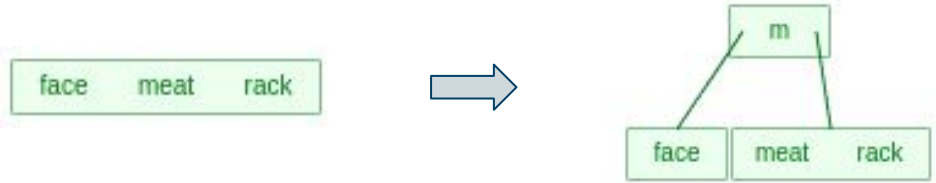
rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

After adding the first three keys, our B-Tree looks like this

Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

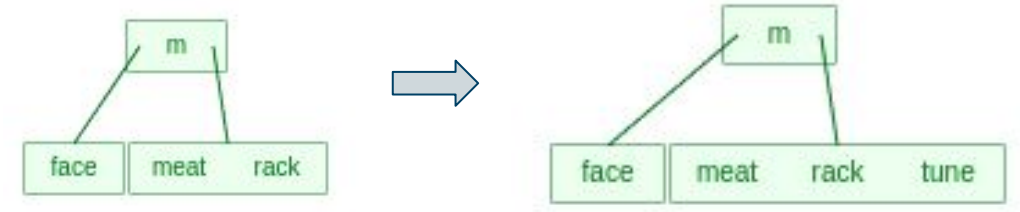
Now we want to add the fourth key (*TUNE*), but the node is full. So, before inserting the new key, a separator must defined. The median key is MEAT. The shortest string between MEAT and FACE is M. So we add M and split the node



Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

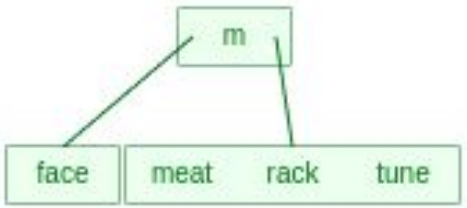
Now we add the following key (*TUNE*)

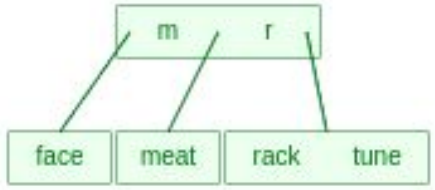


Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

Now we want to add MIST but the node is full. So we take the median key (RACK) and its previous (MEAT), and we use R as separataro. We add R, splitting the node into two nodes

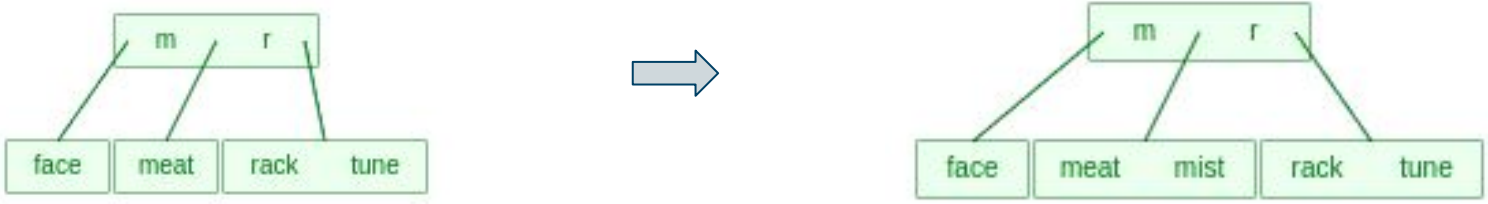




Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

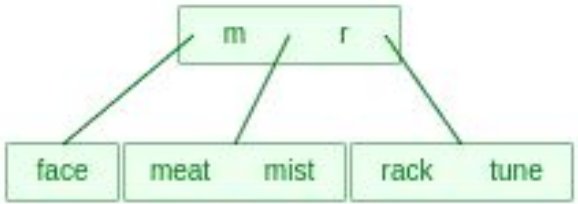
We can now add MIST

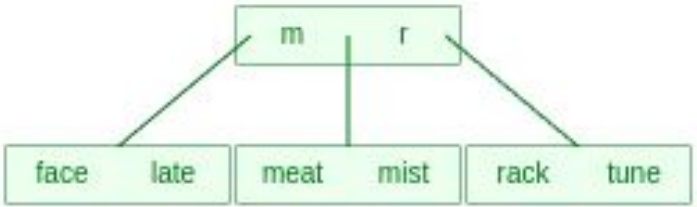


Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

We now want add LATE. The key will fall into the first node, which is not full. So the key can be added

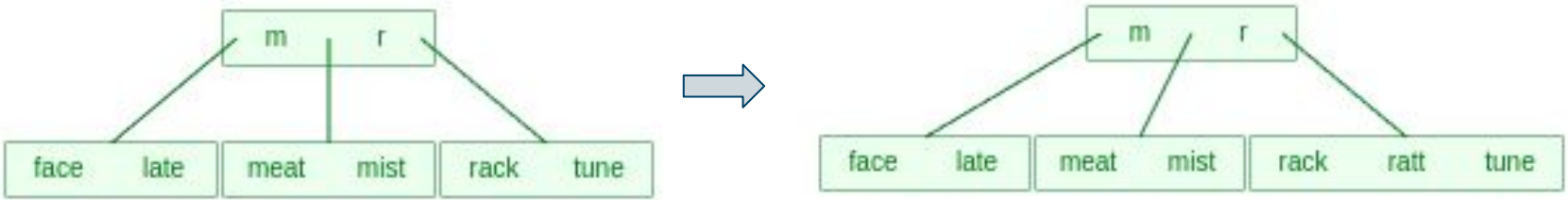




Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

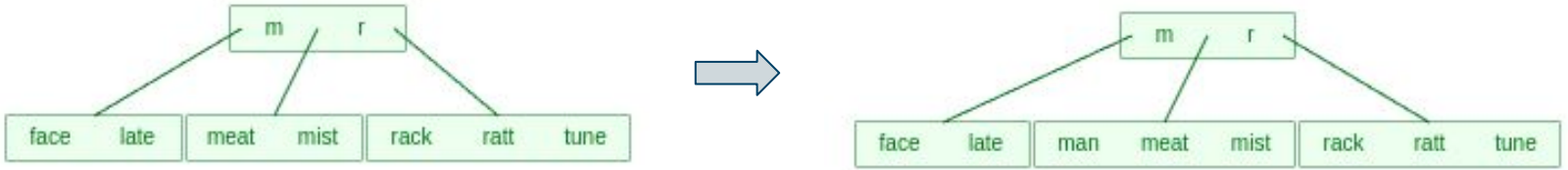
We now want add RATT. The key will fall into the third node, which is not full. So the key can be added.



Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

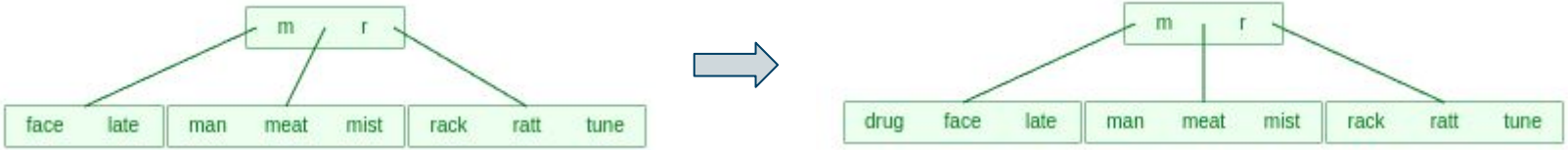
We now want add MAN. The key will fall into the second node, which is not full. So the key can be added.



Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

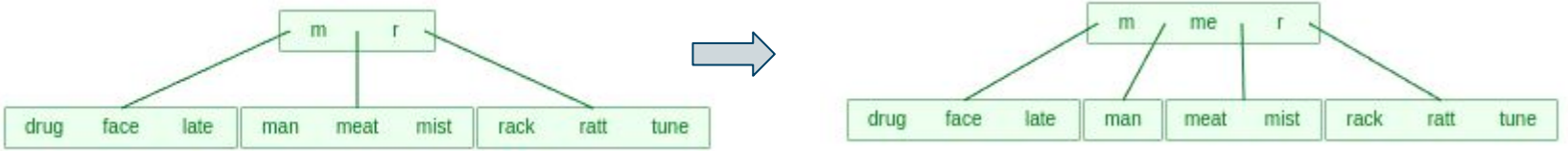
We now want add DRUG. The key will fall into the first node, which is not full. So the key can be added.



Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

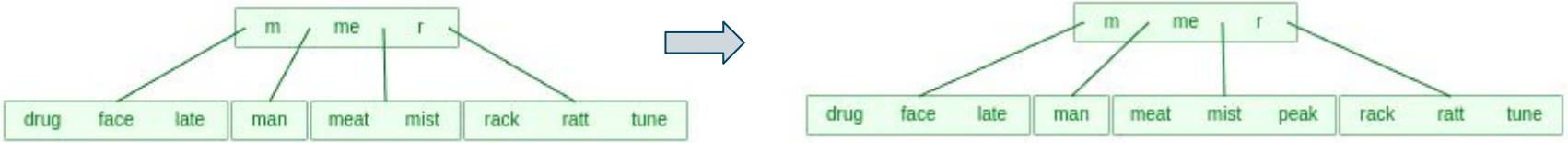
We now want add PEAK. The key will fall into the second node, which is full. So we need to add a separator. The median key is MEAT, the previous key is MAN. The separator is ME. We add ME to the tree and split the second node.



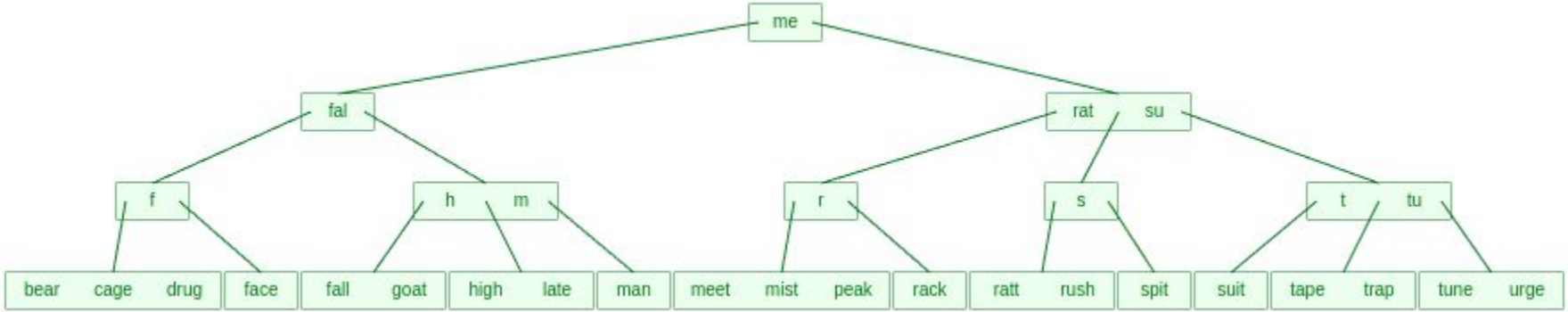
Prefix B-Tree: an example

rack, *face*, *meat*, *tune*, *mist*, *late*, *ratt*, *man*, *drug*, *peak*, *suit*, *fall*, *spit*, *cage*, *high*, *bear*, *goat*, *trap*, *urge*, *rush*, *tape*

We now want add PEAK. The key will fall into the third node, which is not full.



Prefix B-Tree: an example



Sequence of terms and separators: rack, face, meat, m, r, mist, late, ratt, man, drug, me, peak, rat, suit, f, fall, s, spit, cage, fal, high, bear, h, goat, su, trap, t, urge, rush, tu, tape

WildCard Query

WildCard Query

During queries, users can use wildcards (such as *):

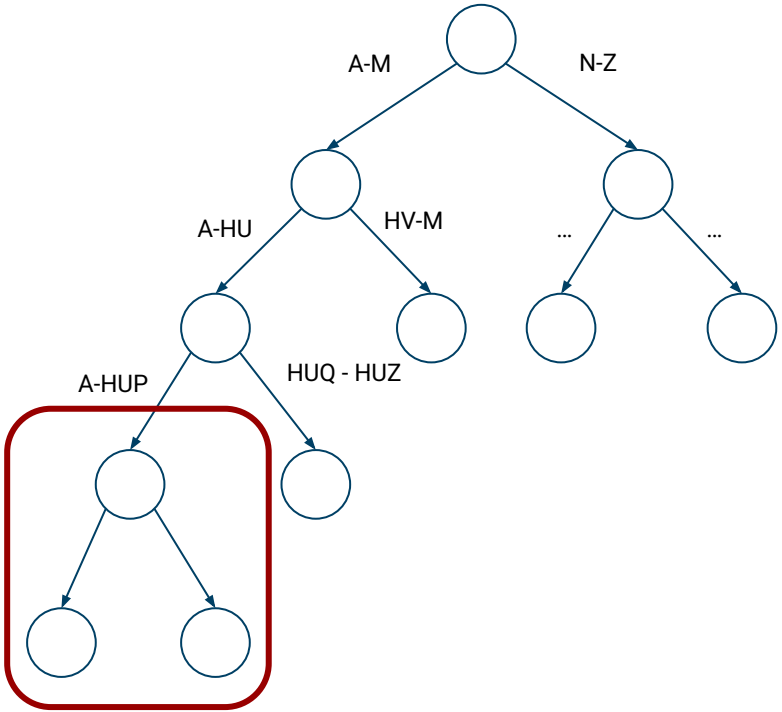
1. when they are uncertain about the spelling of a word (Sidney vs Sydney)
2. when they are aware about the different spelling of a word (color vs colour)
3. when they are searching for all document containing a word beginning with a prefix (judicia* to search for judicial, judiciary, …)

WildCard Query

All Prefix Trees work fine when the wildcard is at the end of a word. For example, if I use

*com**

in a query, it means I'm looking for all documents containing words beginning with *com*. I can walk down the tree, following the symbols *c*, *o* and *m*, then I can get all leaves of such subtree



WildCard Query

But how we can use Search Trees when wildcard is at the beginning of a word? For example

*mon

Solution: using a **Reversed Search Tree**

Reversed Search Tree is a tree where terms in the dictionary are written in **backwards**

For example, the term *lemon* in the B-Tree would be represented by the path *root-n-o-m-e-l*

WildCard Query

A combination of Search Tree and Reversed Search Tree can be used to manage a more general usage of WildCard.

For example, suppose we are looking for the word

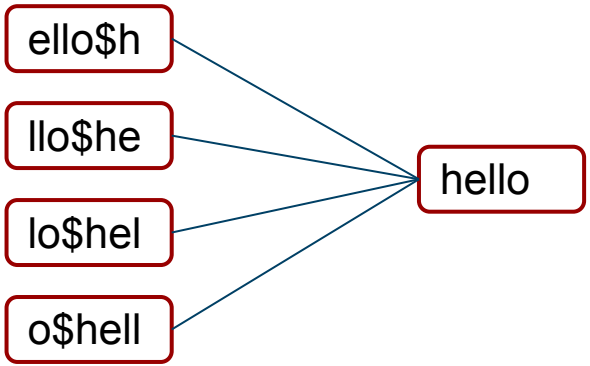
*se*mon*

we can use Search Tree to find all document with a term beginning with *se*, then we can use Reversed Search Tree to find all documents with a term ending with *mon*, then, we can **intersect** all results. Finally we use the **Standard Inverted Index** to retrieve all documents containing any term in this intersection

Permuterm Index

Another solution to manage wildcard queries is by using the **Permuterm Index**

Permuterm index is an index containing all rotations of all terms (augmented by **$**)

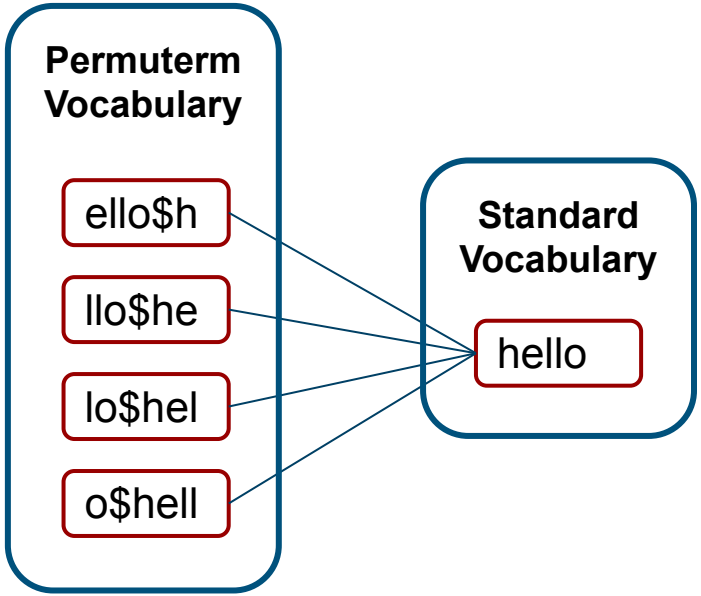


Permuterm Index

All permuterms are stored into a **permuterm vocabulary**

The permuterm index is managed as the same as the **standard inverted index** (for example B-Tree).

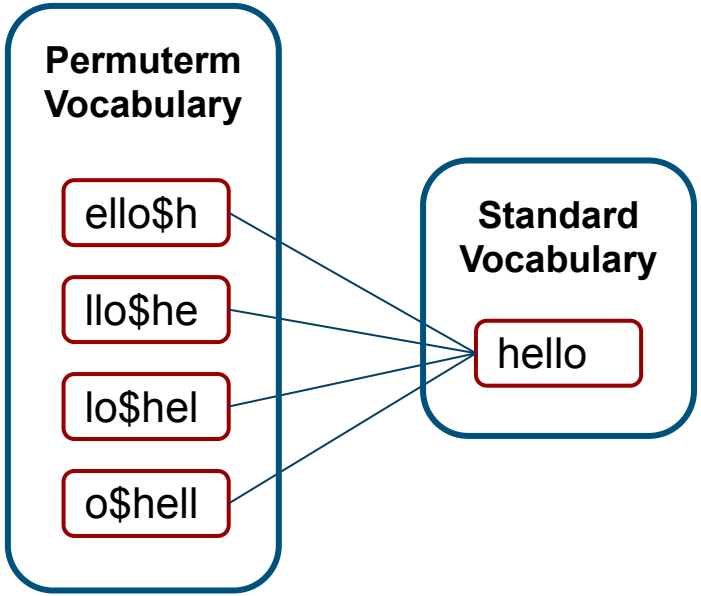
Each term in the permuterm index is **linked to the original vocabulary term**



Permuterm Index

Now let's suppose we are looking for all documents containing *m*n:*

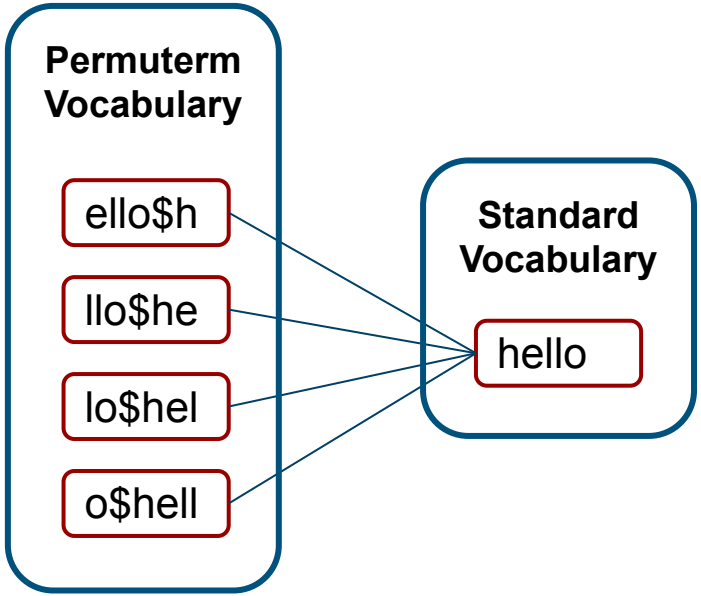
1. we add $ at the end of the query term: *m*n$*
2. we rotate *m*n$* till * is at the end of the term: *n$m**
3. we search into the permuterm index for all terms beginning with *n$m*



Permuterm Index

Now let's suppose we are looking for all documents containing *m*n:*

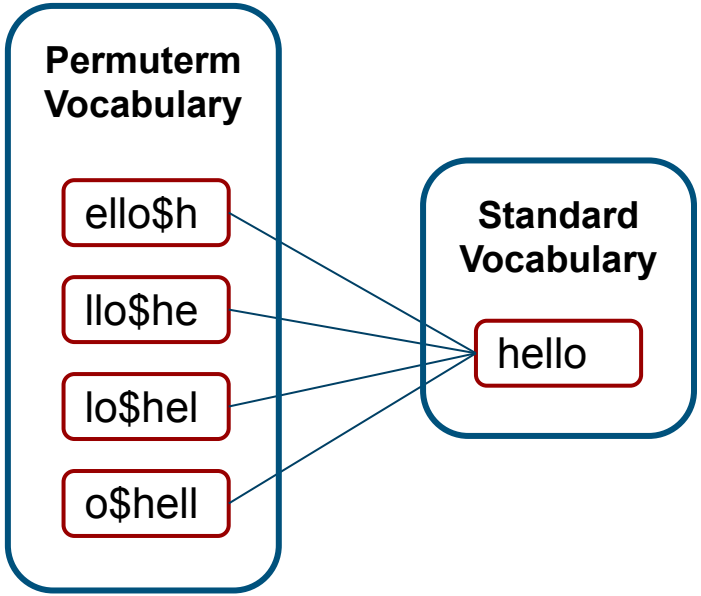
4. we retrieve all permuterms satisfying the constraints
5. by using links, we identify all original terms matching the wildcard (for example, among others, *man* and *moron*)



Permuterm Index

Now let's suppose we are looking for all documents containing *m*n:*

6. we use the original term and by using the standard inverted index, we retrieve all documents containing those terms



Permuterm Index

Ok, this works when * is used in the middle of any term, but what about a query term containing more than one wildcards? Such as *fi*mo*er*?

1. **Filtering Step**: We first look for the query term *fi*er*
   a. add $ at the end of the query term: *fi*er$*
   b. rotate the term till the * is at the end: *er$fi**
   c. search for the permuterm *er$fi* into the permuterm index
   d. retrieve all permuterm satisfying the constraints
   e. identify all original terms matching the wildcard
2. **Post-filtering step**: We look all original terms to see if they contain *mo*
   a. terms not containing *mo* are discarded

Considerations with Permuterm

- Two indexes are required
  - permuterm index and standard inverted index must both exist
- Permuterm vocabulary is quite large, as it includes all rotations of the term
  - an alternative is the **K-gram index**

K-Gram Index

At indexing time, a **k-gram index** is built.

A **k-gram** is a sequence of k characters.

Starting from a term, we can build all k-grams, after adding **$** at the beginning and at the end of the term

Usually k = 3, but it can be whatsoever.

Example:

term

- *castle*

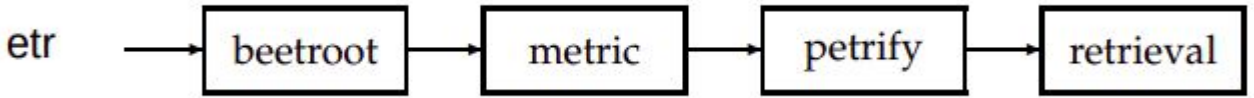
3-grams

- *$ca*
- *cas*
- *ast*
- *stl*
- *tle*
- *le$*

K-Gram Index

A k-gram index contains all k-grams that occur in any term in the vocabulary.

The Postings List of each k-gram points not to the document but to the the term containing that k-gram. For example the 3-gram *etr* appears in four different terms



K-Gram Index

Now, suppose that at query time we are looking for the query ***re*ve***.

Instead of looking for *re*ve*, we can look for those documents containing terms starting with *re* and ending with *ve*, that is: ***$re AND ve$***

This query is run on the k-gram index, which provides a list of terms (for example *relieve*, *remove*, *retrieve*). Each term is then used in the standard inverted index for retrieving documents

K-Gram Index

A problem of the k-gram index.

Suppose we are looking for *red**, we can inquire our k-gram index with the query:

$re AND red

BUT even *retired* is matched, although it should not be.

Solution: using a **post-filtering step**, where each term retrieved from the k-gram index is checked against the original query *red**

Only those terms that survive to the post-filtering step are then searched in the standard inverted index

Spelling correction

Misspelling

Often, users make mistake when they write queries

A good Search Engine should be able to identify mistakes and apply a correction

Spelling Errors are divided into **two categories**:

- non-word misspelling: the error leads to a non existing word (e.g. guitar →guittar)
- real-word misspelling: the error leads to an existing word (e.g. their → there)

Misspelling

For non-word misspelling, usually two techniques are used: **Edit Distance** and **K-Gram Overlap**.

Both methods compute distance between the typed query term and vocabulary terms to see which of them is the **nearest one**

Edit Distance

Given two strings s_1 and s_2, the **edit distance** is the minimum number of edit operations required to transform s_1 into s_2.

Edit Distance is sometimes known as **Levenshtein Distance**.

For edit operation we mean:

- insert a character
- delete a character
- replace a character

Edit Distance

```
EDITDISTANCE(s1, s2)
 1  int m[i, j] = 0
 2  for i ← 1 to |s1|
 3  do m[i, 0] = i
 4  for j ← 1 to |s2|
 5  do m[0, j] = j
 6  for i ← 1 to |s1|
 7  do for j ← 1 to |s2|
 8     do m[i, j] = min{m[i − 1, j − 1] + if (s1[i] = s2[j]) then 0 else 1fi,
 9                      m[i − 1, j] + 1,
10                      m[i, j − 1] + 1}
11  return m[|s1|, |s2|]
```

bidimensional matrix (M+1)x(N+1)
(M = lenght(s_1); N = length(s_2)

the distance to convert the first *i* characters of s_1 into the first *j* characters of s_2

Edit Distance: an example

		F	A	S	T
C					
A					
T					
S					

We want to compute the distance between *fast* and *cats*.

This is the Levenshtein Matrix

(M+1)x(N+1) size

Edit Distance: an example

		F	A	S	T
	0	0	0	0	0
C	0	0	0	0	0
A	0	0	0	0	0
T	0	0	0	0	0
S	0	0	0	0	0

Initialization: all elements are set to 0 (line 1)

```
EDITDISTANCE(s1, s2)
 1  int m[i, j] = 0
 2  for i ← 1 to |s1|
 3  do m[i, 0] = i
 4  for j ← 1 to |s2|
 5  do m[0, j] = j
 6  for i ← 1 to |s1|
 7  do for j ← 1 to |s2|
 8     do m[i, j] = min{m[i − 1, j − 1] + if (s1[i] = s2[j]) then 0 else 1fi,
 9                      m[i − 1, j] + 1,
10                      m[i, j − 1] + 1}
11  return m[|s1|, |s2|]
```

Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	0	0	0	0
A	2	0	0	0	0
T	3	0	0	0	0
S	4	0	0	0	0

Initialization:
first column from 1 to M
first row from 1 to N

```
EDITDISTANCE(s1, s2)
 1  int m[i, j] = 0
 2  for i ← 1 to |s1|
 3  do m[i, 0] = i
 4  for j ← 1 to |s2|
 5  do m[0, j] = j
 6  for i ← 1 to |s1|
 7  do for j ← 1 to |s2|
 8     do m[i, j] = min{m[i − 1, j − 1] + if (s1[i] = s2[j]) then 0 else 1fi,
 9                      m[i − 1, j] + 1,
10                      m[i, j − 1] + 1}
11  return m[|s1|, |s2|]
```

Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	1	0	0	0
A	2	0	0	0	0
T	3	0	0	0	0
S	4	0	0	0	0

Computing distance (lines 6-10):

We compare C with F. They **are different**. So, the value for m(1, 1) =

min(0+1, 1+1, 1+1) = 1

```
EDITDISTANCE(s1, s2)
1  int m[i, j] = 0
2  for i ← 1 to |s1|
3  do m[i, 0] = i
4  for j ← 1 to |s2|
5  do m[0, j] = j
6  for i ← 1 to |s1|
7  do for j ← 1 to |s2|
8     do m[i, j] = min{m[i − 1, j − 1] + if (s1[i] = s2[j]) then 0 else 1fi,
9                      m[i − 1, j] + 1,
10                     m[i, j − 1] + 1}
11 return m[|s1|, |s2|]
```

Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	1	2	0	0
A	2	0	0	0	0
T	3	0	0	0	0
S	4	0	0	0	0

Computing distance (lines 6-10):

We compare C with A. They **are different**. So, the value for m(1, 2) =

min(1+1, 2+1, 1+1) = 2

```
EDITDISTANCE(s1, s2)
1  int m[i, j] = 0
2  for i ← 1 to |s1|
3  do m[i, 0] = i
4  for j ← 1 to |s2|
5  do m[0, j] = j
6  for i ← 1 to |s1|
7  do for j ← 1 to |s2|
8     do m[i, j] = min{m[i − 1, j − 1] + if (s1[i] = s2[j]) then 0 else 1fi,
9                      m[i − 1, j] + 1,
10                     m[i, j − 1] + 1}
11 return m[|s1|, |s2|]
```

Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	1	2	3	4
A	2	2	1	0	0
T	3	0	0	0	0
S	4	0	0	0	0

Computing distance (lines 6-10):

We compare A with A. They **are the same**. So, the value for m(2, 2) =

min(1+**0**, 2+1, 2+1) = 1

```
EDITDISTANCE(s1, s2)
1  int m[i, j] = 0
2  for i ← 1 to |s1|
3  do m[i, 0] = i
4  for j ← 1 to |s2|
5  do m[0, j] = j
6  for i ← 1 to |s1|
7  do for j ← 1 to |s2|
8     do m[i, j] = min{m[i − 1, j − 1] + if (s1[i] = s2[j]) then 0 else 1fi,
9                      m[i − 1, j] + 1,
10                     m[i, j − 1] + 1}
11 return m[|s1|, |s2|]
```

Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	1	2	3	4
A	2	2	1	2	3
T	3	3	2	2	2
S	4	0	0	0	0

Computing distance (lines 6-10):

We compare T with T. They **are the same**. So, the value for m(3, 4) =

min(2+**0**, 2+1, 3+1) = 2

```
EDITDISTANCE(s1, s2)
1  int m[i, j] = 0
2  for i ← 1 to |s1|
3  do m[i, 0] = i
4  for j ← 1 to |s2|
5  do m[0, j] = j
6  for i ← 1 to |s1|
7  do for j ← 1 to |s2|
8     do m[i, j] = min{m[i − 1, j − 1] + if (s1[i] = s2[j]) then 0 else 1fi,
9                      m[i − 1, j] + 1,
10                     m[i, j − 1] + 1}
11 return m[|s1|, |s2|]
```

Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	1	2	3	4
A	2	2	1	2	3
T	3	3	2	2	2
S	4	4	3	2	3

Return distance (line 11):

The returned value is the distance between two words (*fast* and cats)

```
EDITDISTANCE(s1, s2)
 1  int m[i, j] = 0
 2  for i ← 1 to |s1|
 3  do m[i, 0] = i
 4  for j ← 1 to |s2|
 5  do m[0, j] = j
 6  for i ← 1 to |s1|
 7  do for j ← 1 to |s2|
 8     do m[i, j] = min{m[i − 1, j − 1] + if (s1[i] = s2[j]) then 0 else 1fi,
 9                      m[i − 1, j] + 1,
10                      m[i, j − 1] + 1}
11  return m[|s1|, |s2|]
```

Edit Distance: another example

		F	R	I	E	D
	0	1	2	3	4	5
F	1	0	1	2	3	4
R	2	1	0	1	2	3
E	3	2	1	1	1	2
S	4	3	2	2	2	2
H	5	4	3	3	3	3

Distance between FRIED and FRESH: 3

Considerations on Edit Distance

The **complexity** is O(M*N) where M = $|S_1|$ and N = $|S_2|$

The spelling correction problem is more than just computing distance. Indeed having V terms in the Vocabulary, we need to compute the distance between the **query term and each term in the vocabulary**. This takes a lot of time.

Two heuristics are usually used.

Heuristic 1

Statistically, the error **hardly ever occurs at the beginning of the term**. So we can restrict computing distances only with those terms in the vocabulary beginning with the same letters of the query term.

Heuristic 2

Computing distance only with those terms retrieved from a **“refined” Permuterm Index:**

1. Let *q* be the query term
2. Let r_1, r_2, …, r_n be all possible rotations of q
3. Let *l be* an integer, representing the length of the suffix we ignore on each rotation
4. Let t_1, t_2, t_n be all rotations without the last *l* characters
5. For each t_i we look for such a term into the Permuterm Index to find all vocabulary terms
6. Computing distance only with those terms

Heuristic 2

Example:

1. we write **recieve** (instead of **receive**)

2. We compute all rotations:

recieve$	eve$reci
ecieve$r	ve$recie
cieve$re	e$reciev
ieve$rec	

Heuristic 2

Example:

3. we set *l* = 4

4. We get all rotations without the last *l* characters:

reci	eve$
ecie	ve$r
ciev	e$re
ieve	

Heuristic 2

5. For each term we look into the Permuterm Index

```
reci → [appreciated, imprecision, recipe, precise, … ]
eice → [multispecies, species, … ]
ciev → [ ]
ieve → [ achieve, believe, relieve, thieves, … ]
eve$ → [ believe, relieve, … ]
ve$r → [ radioactive, rave, receive, reflective, resolve, … ]
e$re → [ realize, realtime, recharge, receive, recipe, … ]
```

K-Gram Overlap

A second method is named K-Gram Overlap.

This method aims:

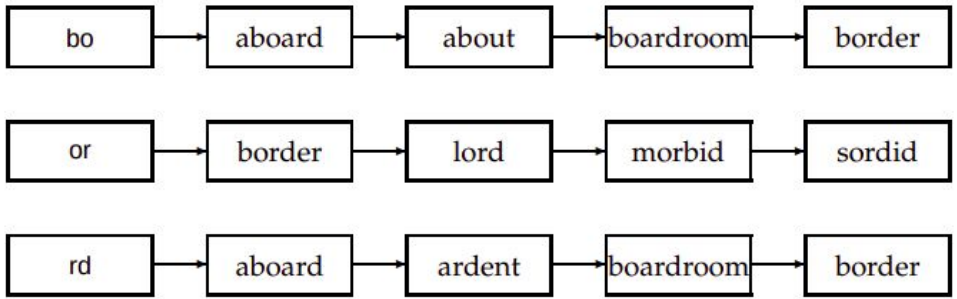
- computing distance between two terms
  - here the distance between two terms describes a measure of **overlapping** (an index to describe how many k-grams are in common)
- reducing the amount of vocabulary terms for which we compute distance

K-Gram Overlap

Suppose we are looking for the misspelled query term *bord*.

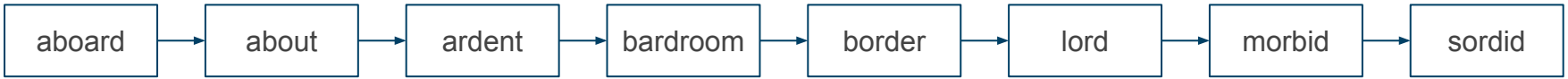
Step 1: build a list of k-grams for the misspelled term (suppose K = 2 we get: *bo*, *or*, *rd*)

Step 2: by using the K-Gram Index, let's retrieve the posting list for all k-grams



K-Gram Overlap

Step 3: merge all retrieved postings lists in one



Step 4: for each term *t* in the merged posting list, we compute the distance between *t* and the query *q*, by using the **Jaccard Similarity**

Step 5: if the coefficient exceeds a preset threshold, we add *t* to the output; if not, we move on to the next term in the postings.

Jaccard Similarity Coefficient

It is a statistical index for comparing similarity between two sets.

Let A and B be two sets, the Jaccard Similarity Coefficient can be computed as

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}.$$

Jaccard Similarity Coefficient is equal to 1 if A and B are the same, 0 if they are different

Combining Jaccard Similarity with K-Grams

Example 1:

q = Query Term: *bord*

t = Vocabulary Term *t*: *aboard*

A = Set of 2-Grams for *q*: {*bo*, *or*, *rd*}

B = Set of 2-Grams for *t*: {*ab, bo, oa, ar, rd*}

J(A, B) = 2/6 = 0.33

Combining Jaccard Similarity with K-Grams

Example 2:

q = Query Term: *bord*

t = Vocabulary Term *t*: *boardroom*

A = Set of 2-Grams for *q*: {*bo*, *or*, *rd*}

B = Set of 2-Grams for *t*: {*bo, oa, ar, rd, dr, ro, oo, om*}

J(A, B) = 2/9 = 0.22

Isolated term correction

Both techniques described so far (**Edit Distance** and **K-gram Overlap**) are real-word misspelling error detectors. They are also named **Isolated Term Correction**.

Suppose for example that we would look for the sentence *fly from Heathrow*, but instead of typing *from* we type *form*. This mistake won't be identified by both method being *form* a non misspelled word.

For such a correction, we need to use another type of methods named **context sensitive correction**

Context Sensitive Correction

The idea is to replace each term *t* in the query *q* with another term *d* which is closer to *t* even though the term *t* is correct.

For each such substitute phrase, the search engine runs the query and determines the number of matching results.

If the number of documents retrieved using the substitute phrase is larger than the original one, the search engine may like to offer the corrected query.

Spelling Suggestion

When the Search Engine realizes that user might have typed the query term in the wrong way, it should suggest to the user the possible query to be used. For example saying: “*Did you mean carrot instead of carot?*”

Phonetic Correction: Soundex Algorithm

Often, terms are misspelled because of their sound. Users typically type a query that sounds like the target term.

For example *cigarette* vs *sigarette*?

To address this problem, we can build an inverted index using the sound as hash key. This kind of index is named **soundex index**

Phonetic Correction: Soundex Algorithm

The following is a typical algorithm for building the **hash key**:

1. Let *t* be the term. Let α be the first character of *t*
2. Replace all characters in *t* with a digit
    a. A, E, I, O, U, H, W, Y ⇒ 0
    b. B, F, P, V ⇒ 1
    c. C, G, J, K, Q, S, X, Z ⇒ 2
    d. D,T ⇒ 3
    e. L ⇒ 4
    f. M, N ⇒ 5
    g. R ⇒ 6

Phonetic Correction: Soundex Algorithm

3. Repeatedly remove one out of each pair of consecutive identical digits.
4. Remove all zeros from the resulting string.
5. Add α at the beginning of the sequence of digits
6. Return the first 4 characters

For example: Apple and Appoll are both mapped into A14.

Phonetic Correction: Metaphone 3

Metaphone 3 is an algorithm which returns a rough approximation of how an English word sounds. Names and Words that are pronounced similarly will have a similar value. It has been release on 2009

There are many implementation. For java see https://commons.apache.org/

Phonetic Correction: Metaphone 3

The Algorithm defines 22 steps:

1. Drop duplicate adjacent letters, except for `C`
2. Drop the first letter if the string begins with `AE`, `GN`, `KN`, `PN` or `WR`
3. Drop `B` if after `M` at the end of the string
4. `C` transforms into
   - `X` if followed by `IA` or `H`
   - `S` if followed by `I`, `E`, or `Y`
   - `K` otherwise
5. `D` transforms into
   - `J` if followed by `GE`, `GY`, or `GI`
   - `T` otherwise

Phonetic Correction: Metaphone 3

6. Drop `G`
   - if followed by `H` and `H` is not at the end or before a vowel
   - if followed by `N` or `NED` and is at the end of the string
7. `G` transforms into
   - `J` if before `I`, `E` or `Y` and is not a `GG`
   - `K` otherwise
8. Drop `H`
   - if after a vowel and not before a vowel
   - if after `C`, `S`, `P`, `T` or `G`
9. Drop `K` if after `C`
10. `PH` transforms into `F`
11. `Q` transforms into `K`
12. `S` transforms into `X` if followed by `H`, `IO` or `IA`

Phonetic Correction: Metaphone 3

13. `T` transforms into `X` if followed by `IA` or `IO`
14. `TH` transforms into `0` (zero)
15. Drop `T` if followed by `CH`
16. `V` transforms into `F`
17. Drop `W` if not followed by a vowel
18. `WH` transforms into `W` if at the beginning of the string
19. `X` transforms into
    - `S` if at the beginning
    - `KS` otherwise
20. Drop `Y` if not followed by a vowel
21. `Z` transforms into `S`
22. Drop all vowels unless it is the beginning character